# Form D - Student Internship - Progress Form

**Progress Report 1 of 3 | Reporting Period: Weeks 1-2**

| | | | |
|---|---|---|---|
| **Student Intern Name** | [Enter Name] | **Student ID and Dept.** | [Student ID] / Computer Science |
| **Host Institution** | [Enter Host Institution] | **Site Supervisor** | [Enter Site Supervisor] |
| **Faculty Supervisor** | [Enter Faculty Supervisor] | **Date** | [DD / MM / YYYY] |
| **Progress Report Number** | ☒ 2nd Wk ☐ 4th Wk ☐ 6th/8th Wk | | |

## a) Tasks Assigned & Performed (Includes assigned duties and completed tasks)

- Reviewed the DeployGuard mini-PaaS objectives, architecture, major modules, and planned AWS deployment workflow.
- Set up the local development environment with Node.js 22, NestJS, React/Vite, PostgreSQL, Docker, Terraform, AWS CLI, Git and GitHub; organized the initial project structure.
- Created and configured the code repository, backend/frontend structure, database entities and migrations, and local environment configuration.
- Implemented user authentication using email/password and GitHub OAuth, authenticated API access, and role-based access control for Admin, Developer and Read-only roles.
- Implemented workspace/project management, including project creation, repository selection and core project CRUD flows.

## b) Learning Experience (Skills and knowledge gained during the internship period)

- Practical understanding of NestJS service/module structure, React frontend integration and PostgreSQL schema/migration management.
- Hands-on experience with GitHub OAuth, authentication flow design, RBAC and secure environment configuration.
- Improved understanding of how a PaaS control plane separates user/project state from later deployment execution.
- Learned to validate local startup, database connectivity and service configuration systematically before feature development.

**Form D - Progress Report 1 of 3 - Weeks 1-2**

**c) Limitations & Challenges (Any challenges, limitations and obstacles while fulfilling the tasks)**

- Initial PostgreSQL port/configuration conflicts and environment-variable mismatches required standardizing local service settings and migration startup.
- GitHub OAuth configuration and callback handling required careful coordination between frontend, backend and GitHub application settings.
- Keeping the project structure modular while several platform modules were being developed in parallel required clear ownership boundaries.
- Some tools listed in the original plan were evaluated early, but the active deployment architecture was refined later as implementation evidence improved.

_____

_____

_____

_____

_____

**Student Intern Signature** _____

# Form D - Student Internship - Progress Form

**Progress Report 2 of 3 | Reporting Period: Weeks 3-4**

| | | | |
|---|---|---|---|
| **Student Intern Name** | [Enter Name] | **Student ID and Dept.** | [Student ID] / Computer Science |
| **Host Institution** | [Enter Host Institution] | **Site Supervisor** | [Enter Site Supervisor] |
| **Faculty Supervisor** | [Enter Faculty Supervisor] | **Date** | [DD / MM / YYYY] |
| **Progress Report Number** | ☐ 2nd Wk ☒ 4th Wk ☐ 6th/8th Wk | | |

## a) Tasks Assigned & Performed (Includes assigned duties and completed tasks)

- Integrated GitHub repository access, branch/commit resolution and repository analysis for authorized projects.
- Implemented Node.js and Python stack detection with mainstream framework detection, package-manager/runtime discovery, application-root resolution and safe Python entrypoint inspection.
- Created an authoritative, versioned Deployment Profile/BuildPlan keyed to repository, branch and commit, containing install/build/run commands, runtime data, ports, outputs and required inputs.
- Implemented Docker template matching and secure Dockerfile generation for supported application modes, including non-root runtime behavior and custom-Dockerfile validation.
- Implemented pre-flight/readiness evaluation with READY, READY_WITH_WARNINGS, INPUT_REQUIRED and BLOCKED outcomes and connected it to the Review Readiness user flow.
- Added regression fixtures and container/runtime smoke checks to verify detector-to-BuildPlan-to-Docker behavior across supported framework modes.

## b) Learning Experience (Skills and knowledge gained during the internship period)

- Learned repository analysis using manifests, lockfiles, framework configuration and bounded structural/AST inspection instead of executing untrusted application code.
- Gained experience designing a single authoritative BuildPlan so later Docker, CI/CD and deployment stages do not independently guess application behavior.
- Improved knowledge of secure Docker image generation, non-root containers, package-manager differences and framework-specific build/start contracts.
- Learned the difference between a platform-solvable repository variation, an input-required ambiguity and a genuine deployment blocker.

**Form D - Progress Report 2 of 3 - Weeks 3-4**

**c) Limitations & Challenges (Any challenges, limitations and obstacles while fulfilling the tasks)**

- Different repository layouts, package managers, framework adapters and monorepo roots made one-size-fits-all detection unreliable.
- Ambiguous entrypoints and incomplete repository evidence had to fail safely without unnecessarily blocking valid applications.
- Missing lockfiles and framework-specific production-runtime differences required normalized BuildPlan behavior while keeping the customer repository unchanged.
- Maintaining consistency across detection, readiness, Docker generation and persisted deployment contracts required extensive regression testing.

_____

_____

_____

_____

_____

**Student Intern Signature** _____

# Form D - Student Internship - Progress Form

**Progress Report 3 of 3 | Reporting Period: Weeks 5-6**

| | | | |
|---|---|---|---|
| **Student Intern Name** | [Enter Name] | **Student ID and Dept.** | [Student ID] / Computer Science |
| **Host Institution** | [Enter Host Institution] | **Site Supervisor** | [Enter Site Supervisor] |
| **Faculty Supervisor** | [Enter Faculty Supervisor] | **Date** | [DD / MM / YYYY] |
| **Progress Report Number** | ☐ 2nd Wk ☐ 4th Wk ☒ 6th/8th Wk | | |

## a) Tasks Assigned & Performed (Includes assigned duties and completed tasks)

- Implemented the active CI/CD execution path using GitHub Actions with immutable reusable-workflow pinning, persisted deployment operations and controlled dispatch from the DeployGuard backend.
- Implemented Docker image build and publication to AWS ECR with immutable commit/image identity and deployment-attempt history.
- Evaluated the originally planned Redis/BullMQ queue approach; the active deployment path was consolidated around GitHub Actions plus PostgreSQL operation state to avoid duplicate deployment executors.
- Integrated Trivy-based vulnerability scanning, scan-result parsing, Docker/security validation and policy-based deployment blockers.
- Added deployment safety gates and approval/readiness checks so unsafe or incomplete application contracts are rejected before infrastructure execution.
- Reviewed the Infracost/cost-estimation and approval-flow requirements and prepared the FinOps integration boundary; final cost-estimation integration remained a continuing implementation item.

## b) Learning Experience (Skills and knowledge gained during the internship period)

- Gained practical experience with GitHub Actions reusable workflows, immutable workflow references, deployment operation tracking, Docker image publication and AWS ECR.
- Learned how to avoid two competing deployment engines by separating authoritative operation state from external CI/CD execution.
- Improved understanding of vulnerability scanning, security policy enforcement, fail-closed deployment gates and sanitizing failure evidence.
- Developed a clearer understanding of infrastructure cost-estimation requirements and how FinOps checks should fit into a deployment approval workflow.

**Form D - Progress Report 3 of 3 - Weeks 5-6**

**c) Limitations & Challenges (Any challenges, limitations and obstacles while fulfilling the tasks)**

- The original Redis/BullMQ worker design overlapped with GitHub Actions execution; keeping both active would risk duplicate jobs, so the architecture was consolidated to one authoritative execution path.
- CI/CD reliability required careful workflow pinning, immutable operation identity and consistent handoff of repository/BuildPlan data.
- Security scanners can produce noisy results, so policy thresholds had to distinguish blocking vulnerabilities from advisory findings.
- Cost estimation/approval integration was broader than the security work and required additional infrastructure-plan evidence, so it remained the main Week 6 item requiring continued integration.

_____

_____

_____

_____

_____

**Student Intern Signature** _____